5.4 Efficiency of Python's Sequence Types

In the previous section, we began to explore the underpinnings of Python's list class, in terms of implementation strategies and efficiency. We continue in this section by examining the performance of all of Python's sequence types.

5.4.1 Python's List and Tuple Classes

The *nonmutating* behaviors of the list class are precisely those that are supported by the tuple class. We note that tuples are typically more memory efficient than lists because they are immutable; therefore, there is no need for an underlying dynamic array with surplus capacity. We summarize the asymptotic efficiency of the nonmutating behaviors of the list and tuple classes in Table 5.3. An explanation of this analysis follows.

Operation	Running Time
len(data)	$O(1)$
data[j]	$O(1)$
data.count(value)	$O(n)$
data.index(value)	$O(k+1)$
value **in** data	$O(k+1)$
data1 == data2 (similarly !=, <, <=, >, >=)	$O(k+1)$
data[j:k]	$O(k-j+1)$
data1 + data2	$O(n_1+n_2)$
c * data	$O(cn)$

Table 5.3: Asymptotic performance of the nonmutating behaviors of the list and tuple classes. Identifiers data, data1, and data2 designate instances of the list or tuple class, and n, n_1, and n_2 their respective lengths. For the containment check and index method, k represents the index of the leftmost occurrence (with $k = n$ if there is no occurrence). For comparisons between two sequences, we let k denote the leftmost index at which they disagree or else $k = \min(n_1, n_2)$.

Constant-Time Operations

The length of an instance is returned in constant time because an instance explicitly maintains such state information. The constant-time efficiency of syntax data[j] is assured by the underlying access into an array.

Searching for Occurrences of a Value

Each of the count, index, and __contains__ methods proceed through iteration of the sequence from left to right. In fact, Code Fragment 2.14 of Section 2.4.3 demonstrates how those behaviors might be implemented. Notably, the loop for computing the count must proceed through the entire sequence, while the loops for checking containment of an element or determining the index of an element immediately exit once they find the leftmost occurrence of the desired value, if one exists. So while count always examines the n elements of the sequence, index and __contains__ examine n elements in the worst case, but may be faster. Empirical evidence can be found by setting data = list(range(10000000)) and then comparing the relative efficiency of the test, 5 **in** data, relative to the test, 9999995 **in** data, or even the failed test, -5 **in** data.

Lexicographic Comparisons

Comparisons between two sequences are defined lexicographically. In the worst case, evaluating such a condition requires an iteration taking time proportional to the length of the *shorter* of the two sequences (because when one sequence ends, the lexicographic result can be determined). However, in some cases the result of the test can be evaluated more efficiently. For example, if evaluating [7, 3, ...] $<$ [7, 5, ...], it is clear that the result is True without examining the remainders of those lists, because the second element of the left operand is strictly less than the second element of the right operand.

Creating New Instances

The final three behaviors in Table 5.3 are those that construct a new instance based on one or more existing instances. In all cases, the running time depends on the construction and initialization of the new result, and therefore the asymptotic behavior is proportional to the *length* of the result. Therefore, we find that slice data[6000000:6000008] can be constructed almost immediately because it has only eight elements, while slice data[6000000:7000000] has one million elements, and thus is more time-consuming to create.

Mutating Behaviors

The efficiency of the mutating behaviors of the list class are described in Table 5.3. The simplest of those behaviors has syntax data[j] = val, and is supported by the special __setitem__ method. This operation has worst-case $O(1)$ running time because it simply replaces one element of a list with a new value. No other elements are affected and the size of the underlying array does not change. The more interesting behaviors to analyze are those that add or remove elements from the list.

Operation	Running Time
data[j] = val	$O(1)$
data.append(value)	$O(1)^*$
data.insert(k, value)	$O(n-k+1)^*$
data.pop( )	$O(1)^*$
data.pop(k) **del** data[k]	$O(n-k)^*$
data.remove(value)	$O(n)^*$
data1.extend(data2) data1 += data2	$O(n_2)^*$
data.reverse( )	$O(n)$
data.sort( )	$O(n \log n)$

*amortized

Table 5.4: Asymptotic performance of the mutating behaviors of the list class. Identifiers data, data1, and data2 designate instances of the list class, and n, n_1, and n_2 their respective lengths.

Adding Elements to a List

In Section 5.3 we fully explored the append method. In the worst case, it requires $\Omega(n)$ time because the underlying array is resized, but it uses $O(1)$ time in the amortized sense. Lists also support a method, with signature insert(k, value), that inserts a given value into the list at index $0 \leq k \leq n$ while shifting all subsequent elements back one slot to make room. For the purpose of illustration, Code Fragment 5.5 provides an implementation of that method, in the context of our DynamicArray class introduced in Code Fragment 5.3. There are two complicating factors in analyzing the efficiency of such an operation. First, we note that the addition of one element may require a resizing of the dynamic array. That portion of the work requires $\Omega(n)$ worst-case time but only $O(1)$ amortized time, as per append. The other expense for insert is the shifting of elements to make room for the new item. The time for

```
1  def insert(self, k, value):
2    """Insert value at index k, shifting subsequent values rightward."""
3    # (for simplicity, we assume 0 <= k <= n in this verion)
4    if self._n == self._capacity:                # not enough room
5      self._resize(2 * self._capacity)           # so double capacity
6    for j in range(self._n, k, -1):              # shift rightmost first
7      self._A[j] = self._A[j-1]
8    self._A[k] = value                           # store newest element
9    self._n += 1
```

Code Fragment 5.5: Implementation of insert for our DynamicArray class.

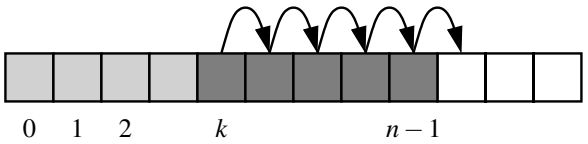


Figure 5.16: Creating room to insert a new element at index k of a dynamic array.

that process depends upon the index of the new element, and thus the number of other elements that must be shifted. That loop copies the reference that had been at index $n-1$ to index n, then the reference that had been at index $n-2$ to $n-1$, continuing until copying the reference that had been at index k to $k+1$, as illustrated in Figure 5.16. Overall this leads to an amortized $O(n-k+1)$ performance for inserting at index k.

When exploring the efficiency of Python's append method in Section 5.3.3, we performed an experiment that measured the average cost of repeated calls on varying sizes of lists (see Code Fragment 5.4 and Table 5.2). We have repeated that experiment with the insert method, trying three different access patterns:

- In the first case, we repeatedly insert at the beginning of a list,

```
for n in range(N):
  data.insert(0, None)
```

- In a second case, we repeatedly insert near the middle of a list,

```
for n in range(N):
  data.insert(n // 2, None)
```

- In a third case, we repeatedly insert at the end of the list,

```
for n in range(N):
  data.insert(n, None)
```

The results of our experiment are given in Table 5.5, reporting the *average* time per operation (not the total time for the entire loop). As expected, we see that inserting at the beginning of a list is most expensive, requiring linear time per operation. Inserting at the middle requires about half the time as inserting at the beginning, yet is still $\Omega(n)$ time. Inserting at the end displays $O(1)$ behavior, akin to append.

	N				
	100	1,000	10,000	100,000	1,000,000
k = 0	0.482	0.765	4.014	36.643	351.590
k = n // 2	0.451	0.577	2.191	17.873	175.383
k = n	0.420	0.422	0.395	0.389	0.397

Table 5.5: Average running time of insert(k, val), measured in microseconds, as observed over a sequence of N calls, starting with an empty list. We let n denote the size of the current list (as opposed to the final list).

Removing Elements from a List

Python's list class offers several ways to remove an element from a list. A call to pop( ) removes the last element from a list. This is most efficient, because all other elements remain in their original location. This is effectively an $O(1)$ operation, but the bound is amortized because Python will occasionally shrink the underlying dynamic array to conserve memory.

The parameterized version, pop(k), removes the element that is at index $k < n$ of a list, shifting all subsequent elements leftward to fill the gap that results from the removal. The efficiency of this operation is $O(n-k)$, as the amount of shifting depends upon the choice of index k, as illustrated in Figure 5.17. Note well that this implies that pop(0) is the most expensive call, using $\Omega(n)$ time. (see experiments in Exercise R-5.8.)

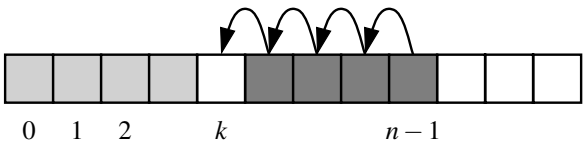


Figure 5.17: Removing an element at index k of a dynamic array.

The list class offers another method, named remove, that allows the caller to specify the *value* that should be removed (not the *index* at which it resides). Formally, it removes only the first occurrence of such a value from a list, or raises a ValueError if no such value is found. An implementation of such behavior is given in Code Fragment 5.6, again using our DynamicArray class for illustration.

Interestingly, there is no "efficient" case for remove; every call requires $\Omega(n)$ time. One part of the process searches from the beginning until finding the value at index k, while the rest iterates from k to the end in order to shift elements leftward. This linear behavior can be observed experimentally (see Exercise C-5.24).

```
1   def remove(self, value):
2     """Remove first occurrence of value (or raise ValueError)."""
3     # note: we do not consider shrinking the dynamic array in this version
4     for k in range(self._n):
5       if self._A[k] == value:                    # found a match!
6         for j in range(k, self._n - 1):          # shift others to fill gap
7           self._A[j] = self._A[j+1]
8         self._A[self._n - 1] = None              # help garbage collection
9         self._n -= 1                             # we have one less item
10        return                                   # exit immediately
11    raise ValueError('value not found')          # only reached if no match
```

Code Fragment 5.6: Implementation of remove for our DynamicArray class.

Extending a List

Python provides a method named extend that is used to add all elements of one list to the end of a second list. In effect, a call to data.extend(other) produces the same outcome as the code,

```
for element in other:
  data.append(element)
```

In either case, the running time is proportional to the length of the other list, and amortized because the underlying array for the first list may be resized to accommodate the additional elements.

In practice, the extend method is preferable to repeated calls to append because the constant factors hidden in the asymptotic analysis are significantly smaller. The greater efficiency of extend is threefold. First, there is always some advantage to using an appropriate Python method, because those methods are often implemented natively in a compiled language (rather than as interpreted Python code). Second, there is less overhead to a single function call that accomplishes all the work, versus many individual function calls. Finally, increased efficiency of extend comes from the fact that the resulting size of the updated list can be calculated in advance. If the second data set is quite large, there is some risk that the underlying dynamic array might be resized multiple times when using repeated calls to append. With a single call to extend, at most one resize operation will be performed. Exercise C-5.22 explores the relative efficiency of these two approaches experimentally.

Constructing New Lists

There are several syntaxes for constructing new lists. In almost all cases, the asymptotic efficiency of the behavior is linear in the length of the list that is created. However, as with the case in the preceding discussion of extend, there are significant differences in the practical efficiency.

Section 1.9.2 introduces the topic of ***list comprehension***, using an example such as squares = [k*k **for** k **in** range(1, n+1)] as a shorthand for

```
squares = [ ]
for k in range(1, n+1):
  squares.append(k*k)
```

Experiments should show that the list comprehension syntax is significantly faster than building the list by repeatedly appending (see Exercise C-5.23).

Similarly, it is a common Python idiom to initialize a list of constant values using the multiplication operator, as in [0] * n to produce a list of length n with all values equal to zero. Not only is this succinct for the programmer; it is more efficient than building such a list incrementally.

5.4.2 Python's String Class

Strings are very important in Python. We introduced their use in Chapter 1, with a discussion of various operator syntaxes in Section 1.3. A comprehensive summary of the named methods of the class is given in Tables A.1 through A.4 of Appendix A. We will not formally analyze the efficiency of each of those behaviors in this section, but we do wish to comment on some notable issues. In general, we let n denote the length of a string. For operations that rely on a second string as a pattern, we let m denote the length of that pattern string.

The analysis for many behaviors is quite intuitive. For example, methods that produce a new string (e.g., capitalize, center, strip) require time that is linear in the length of the string that is produced. Many of the behaviors that test Boolean conditions of a string (e.g., islower) take $O(n)$ time, examining all n characters in the worst case, but short circuiting as soon as the answer becomes evident (e.g., islower can immediately return False if the first character is uppercased). The comparison operators (e.g., $==$, $<$) fall into this category as well.

Pattern Matching

Some of the most interesting behaviors, from an algorithmic point of view, are those that in some way depend upon finding a string pattern within a larger string; this goal is at the heart of methods such as __contains__, find, index, count, replace, and split. String algorithms will be the topic of Chapter 13, and this particular problem known as ***pattern matching*** will be the focus of Section 13.2. A naive implementation runs in $O(mn)$ time case, because we consider the $n-m+1$ possible starting indices for the pattern, and we spend $O(m)$ time at each starting position, checking if the pattern matches. However, in Section 13.2, we will develop an algorithm for finding a pattern of length m within a longer string of length n in $O(n)$ time.

Composing Strings

Finally, we wish to comment on several approaches for composing large strings. As an academic exercise, assume that we have a large string named document, and our goal is to produce a new string, letters, that contains only the alphabetic characters of the original string (e.g., with spaces, numbers, and punctuation removed). It may be tempting to compose a result through repeated concatenation, as follows.

```
# WARNING: do not do this
letters = ''                          # start with empty string
for c in document:
  if c.isalpha( ):
    letters += c                      # concatenate alphabetic character
```

While the preceding code fragment accomplishes the goal, it may be terribly inefficient. Because strings are immutable, the command, letters += c, would presumably compute the concatenation, letters + c, as a new string instance and then reassign the identifier, letters, to that result. Constructing that new string would require time proportional to its length. If the final result has n characters, the series of concatenations would take time proportional to the familiar sum $1+2+3+\cdots+n$, and therefore $O(n^2)$ time.

Inefficient code of this type is widespread in Python, perhaps because of the somewhat natural appearance of the code, and mistaken presumptions about how the += operator is evaluated with strings. Some later implementations of the Python interpreter have developed an optimization to allow such code to complete in linear time, but this is not guaranteed for all Python implementations. The optimization is as follows. The reason that a command, letters += c, causes a new string instance to be created is that the original string must be left unchanged if another variable in a program refers to that string. On the other hand, if Python knew that there were no other references to the string in question, it could implement += more efficiently by directly mutating the string (as a dynamic array). As it happens, the Python interpreter already maintains what are known as ***reference counts*** for each object; this count is used in part to determine if an object can be garbage collected. (See Section 15.1.2.) But in this context, it provides a means to detect when no other references exist to a string, thereby allowing the optimization.

A more standard Python idiom to guarantee linear time composition of a string is to use a temporary list to store individual pieces, and then to rely on the join method of the str class to compose the final result. Using this technique with our previous example would appear as follows:

```
temp = [ ]                          # start with empty list
for c in document:
  if c.isalpha( ):
    temp.append(c)                  # append alphabetic character
letters = ''.join(temp)             # compose overall result
```

This approach is guaranteed to run in $O(n)$ time. First, we note that the series of up to n append calls will require a total of $O(n)$ time, as per the definition of the amortized cost of that operation. The final call to join also guarantees that it takes time that is linear in the final length of the composed string.

As we discussed at the end of the previous section, we can further improve the practical execution time by using a list comprehension syntax to build up the temporary list, rather than by repeated calls to append. That solution appears as,

```
letters = ''.join([c for c in document if c.isalpha( )])
```

Better yet, we can entirely avoid the temporary list with a generator comprehension:

```
letters = ''.join(c for c in document if c.isalpha( ))
```